

Optimizing Gaussian Splat Rendering for Mobile and Standalone VR

Olivier Therrien - CDRIN



SIGGRAPH 2025

Vancouver+ 10-14 August

Olivier Therrien

CDRIN (Centre de Développement et de Recherche en Intelligence Numérique):

R&D Programmer (rendering), GI, Path Tracing, PBR

<https://www.cdrin.com/realisations/>

Ubisoft: 3D Programmer, Assassin's Creed Odyssey, Rainbow Six Siege

Frima FX (Volta): R&D graphics programmer (research real-time GPU path tracer)

Sarbakan: Shader & gameplay programmer

3D Gaussian Splatting

Represents a 3D scene as a set of 3D anisotropic Gaussians (position, orientation, scale, color, and opacity)

Training is fully differentiable – optimizes Gaussians parameters from multi-view images via gradient descent

Rendering is done by projecting the 3D gaussians front-to-back or back-to-front onto the image plane using alpha blending

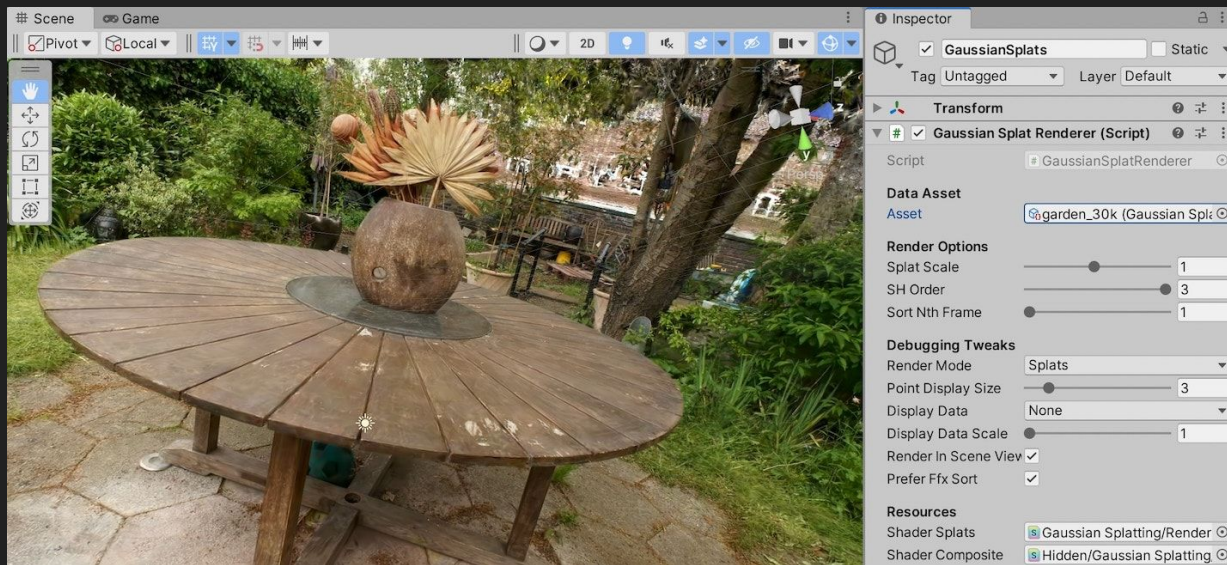
<https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>

Credits: B Kerbl, G Kopanas, T Leimkühler, G Drettakis



UnityGaussianSplatting

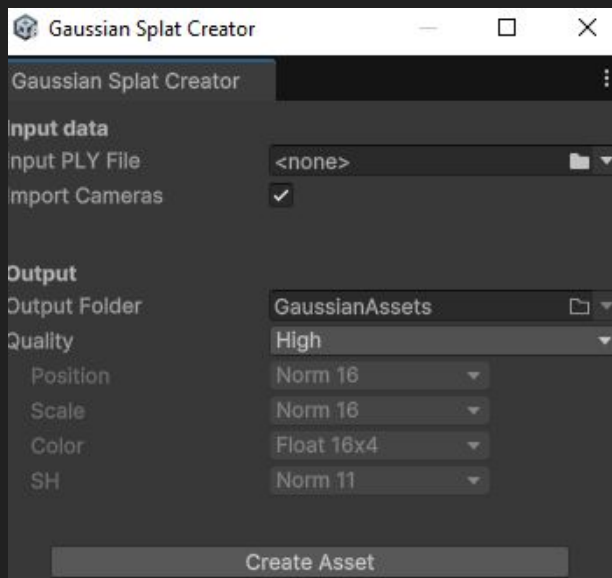
Link: <https://github.com/aras-p/UnityGaussianSplatting>



Credits: Aras Pranckevičius et al.

Generating a gaussian splat asset

Tools -> Gaussian Splats -> Create GaussianSplatAsset



Splats	328,894
Version	20231020
Memory	15.3 MB
Positions	1.3 MB (Norm11)
Other	2.5 MB (Norm11)
Base color	1.4 MB (Norm8x4)
SHs	10.0 MB (Norm6)
Chunks	80.3 KB (64 B/chunk)
Bounds Min	X -69.64624 Y -40.51622 Z -72.28538
Bounds Max	X 55.21032 Y 14.83024 Z 87.48445
Data Hash	f58df01b5919c862b5e6bb1951a084c4



Rendering overview

The renderer works as follows:

1. Sort the splats in a compute shader (radix sort)
2. Draw a quad per splat (instanced)
3. Alpha falloff in the pixel shader

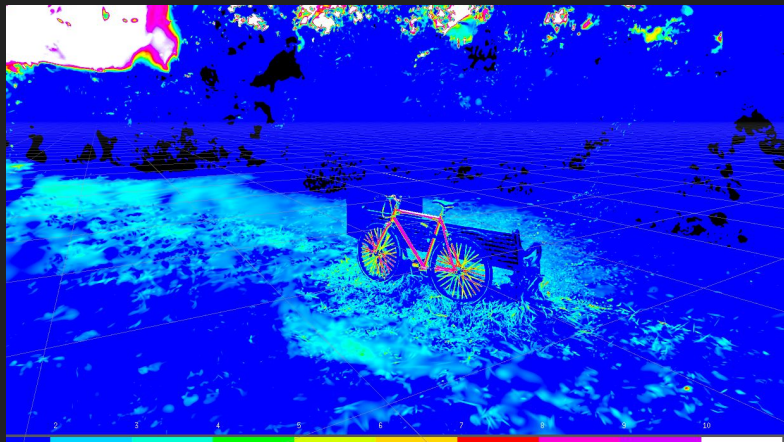
There is a lot of overdraw:



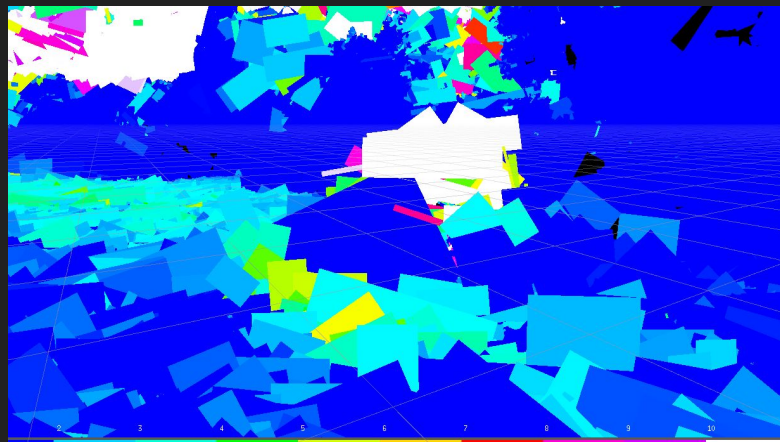
Overdraw

- Drawing multiple times over a pixel (expensive)
- Happens a lot with gaussian splats, even in transparent regions

Splats with alpha multiply:



Splats without alpha:



RenderDoc (Meta Fork)

Download: <https://developer.oculus.com/downloads/package/renderdoc-oculus/>

New Quest-specific functionalities:

- Performance Counters:
<https://developer.oculus.com/documentation/web/ts-webxr-perf-drawcall/>
- Timings (Profiler) with on-device replay
- Tile Browser
- Vulkan shader stats from Adreno driver (KHR_pipeline_executable_properties)

Performance on Quest 3

Timings from RenderDoc:

▼ RenderGaussianSplats	10036.92683829
> GaussianSplat.Sort	127.34374689
> GaussianSplat.CalcView	0.00
> GaussianSplat.Draw	0.00
> GaussianSplat.Compose	9909.5830914

Performance counters suggest lots of overdraw and low shader utilization:

Capture counters  							
EID	GPU Duration (ms)	PS Invocations	% Texture Fetch Stall	% Texture L1 Miss	% Texture L2 Miss	% Stalled on System Memory	% Shaders Busy
176	0.00015625	3156	0.39916241	3.5062089	0.00	0.00	15.56406212
346	0.00015625	3204	0.35507792	3.45572352	0.00	0.00	15.39404202
175	0.00015625	3156	0.03206638	3.5062089	33.33333206	0.00	8.5630331
345	0.00015625	3204	0.02446	3.45572352	33.33333206	0.00	8.34901237
240	0.0084375	0	0.00	0.00	0.00	0.00	3.62230206
70	0.00848958	0	0.00	0.00	0.00	0.00	3.54128575
143	0.00041667	83025696	0.00	0.00	0.00	12.21467209	0.490852
313	0.00057292	81740764	0.00	0.00	0.00	16.01755905	0.46071774

Overdraw

RenderDoc allows for inspecting individual pixels

Can see what happens to each fragment (depth clipped, discarded, drawn, etc)

Some pixels have a very high overdraw count:

- 154 fragments are discarded in the PS
- 55 fragments are drawn to screen

The baseline is 44 ms GPU (OVR Metrics Tool)

Event	Tex Before	Tex After
> RenderGaussianSplats > GaussianSplat.Sort EID 60 vkCmdBeginRenderPass2(C=Clear, DS=Load) 1 Fragments touching pixel	R: 0.96533203 G: 0.96923828 B: 0.97021484 A: 0.99853516 D: 0.00 S: 0x00	R: 0.00 G: 0.00 B: 0.00 A: 0.00 D: 0.00 S: 0x00
> RenderGaussianSplats > GaussianSplat.Draw > EID 143 vkCmdDrawIndexed(6, 328894) 55 Fragments touching pixel	R: 0.00 G: 0.00 B: 0.00 A: 0.00 D: 5.36441860E-7 S: 0x00	R: 0.32226563 G: 0.36425781 B: 0.14880371 A: 0.95996094 D: 5.36441860E-7 S: 0x00
> RenderGaussianSplats > GaussianSplat.Sort EID 230 vkCmdBeginRenderPass2(C=Clear, DS=Load) 1 Fragments touching pixel	R: 0.32226563 G: 0.36425781 B: 0.14880371 A: 0.95996094 D: 0.00 S: 0x00	R: 0.00 G: 0.00 B: 0.00 A: 0.00 D: 0.00 S: 0x00
> RenderGaussianSplats > GaussianSplat.Draw > EID 313 vkCmdDrawIndexed(6, 328894) Shader executed a discard 154 Fragments touching pixel	R: 0.00 G: 0.00 B: 0.00 A: 0.00 D: 5.36441860E-7 S: 0x00	R: 0.00 G: 0.00 B: 0.00 A: 0.00 D: 0.00 S: 0x00

Optimization 1: More aggressive alpha clipping

The pixel shader discards fragments that are fully transparent

When discarding: the pixel shader is still executed, but the GPU doesn't have to do alpha blending (output merger stage)

More aggressive alpha clip threshold value removes some barely visible pixels

```
half4 frag (v2f i) : SV_Target
{
    float power = -dot(i.pos, i.pos);
    half alpha = exp(power);
    if (i.col.a >= 0)
    {
        alpha = saturate(alpha * i.col.a);
    }
    else
    {
        // "selected" splat: magenta outline, increase opacity, magenta tint
        half3 selectedColor = half3(1,0,1);
        if (alpha > 7.0/255.0)
        {
            if (alpha < 10.0/255.0)
            {
                alpha = 1;
                i.col.rgb = selectedColor;
            }
            alpha = saturate(alpha + 0.3);
        }
        i.col.rgb = lerp(i.col.rgb, selectedColor, 0.5);
    }

    if (alpha < 1.0/255.0)
    {
        discard;
    }

    half4 res = half4(i.col.rgb * alpha, alpha);
    return res;
}
```

Optimization 1: More aggressive alpha clipping

There can be a slight visual difference, tweak the value to keep it barely noticeable.

Clip: 1/255, Layers: 80, Perf: ~10 ms



Clip: 15/255, Layers: 25, Perf: 5.5 ms



Clip 25/255, Layers: 15, Perf: 4.7 ms



At 15/255 clipping value, there is a 18 ms improvement over baseline

Optimization 2: Stencil Testing

Increment a counter each time a pixel is drawn

Stop drawing after X layers to limit overdraw of high fragment count pixels (with a max of 9 layers, it gives a 3 ms improvement)

```
Stencil {  
  Ref 25  
  Comp GEQual  
  Pass IncrSat  
}
```

Setting a low value for layer count can cause artifacts:



Optimization 3: Quad rescaling

Resize the quads to fit more closely the shape of the visible portion of the splats:

```
#define QUAD_SCALING_XY float2(0.7, 0.6)

float2 deltaScreenPos = (quadPos.x * view.axis1 + quadPos.y * view.axis2) * 2 / _ScreenParams.xy;
o.vertex = centerClipPos;
o.vertex.xy += deltaScreenPos * centerClipPos.w * QUAD_SCALING_XY;

void frag (v2f i, out half4 ocol : SV_Target)
{
    float power = -dot(i.pos*QUAD_SCALING_XY, i.pos*QUAD_SCALING_XY);

    ...
}
```

This gives another 4 ms improvement

Optimization 4: RTGS (Real-Time Gaussian Splatting)

Better placement of splats to keep detail and limit overlap. (3.5 ms improvement)



Source: <https://arxiv.org/abs/2407.00435>

Optimization 5: Sort only once per frame

In VR the radix sort is done once per eye

This can easily be done only once per frame without any impact on quality

```
if (gs.m_FrameCounter % gs.m_SortNthFrame == 0 && eyeIndex == 0)
    gs.SortPoints(cmb, cam, matrix);
```

It's also possible to sort only once every N frames to improve performance even more

Overall this gives another 2.5 ms improvement

Conclusion

We showed how to optimize the graphics pipeline for faster gaussian splat rendering:

- Alpha clipping to alleviate the blending units
- Stencil test to limit the maximum number of layer
- Quad fitting to reduce pixel shaders invocations
- RTGS for better splat placement and overlap reduction
- Sort only once per eye, and not every frame

With all these improvements, the performance go from 44 ms down to 13 ms, giving a 3.3x boost on mobile and VR, making it usable in real-time.